TALLER DE BASE DE DATOS

Laboratorio 1

Red de clínicas veterinarias

Arquitectura multicapa, PostgreSQL y seguridad JWT

- Anthony Huahuisa
- Francisco Briones
- Juan Campos A
- Nicolas Orellana
- Melissa Shimada
- Ignacio Caro

Universidad de Santiago de Chile · Ingeniería Informática · 2-2026

Problema, usuarios y alcance

Problema de gestión

Coordinar citas, historial clínico, vacunas y farmacia entre sedes.

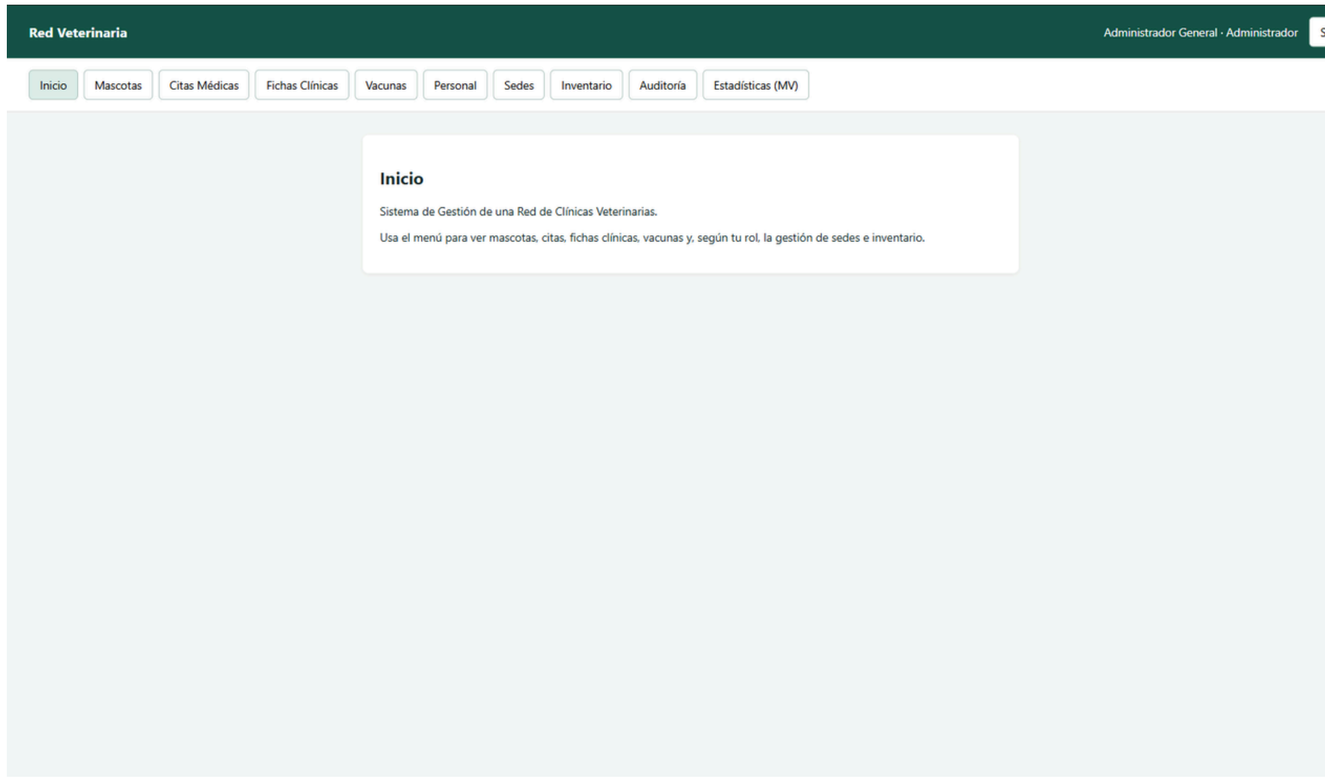
Usuarios del sistema

Tutor, veterinario y administrador; cada uno ve y hace solo lo suyo.

Alcance del laboratorio

PostgreSQL(Base de datos), API REST(Backend) e interfaz web basica(Frontend)





Avance del proyecto

Componente	Qué incluye
Modelo y datos	17 tablas, datos de prueba y dump de la base
API y consultas	CRUD de todas las tablas, paginación y consulta por id
Frontend	Pantallas por rol: citas, fichas, vacunas, inventario y personal
Seguridad	JWT, bcrypt, roles y filtros por tutor y por veterinario
SQL avanzado	Se cumple con el minimo pedido de 2 procedimientos, 2 triggers, 3 índices y 1 vista materializada

Arquitectura multicapa

Frontend web

Interfaz de usuario
React 19 + Vite 7

HTTP / JSON

API REST

Autenticación y RBAC
Express 5 + pg · sin ORM

SQL

PostgreSQL

17 tablas relacionadas
Procedimientos, triggers y vista

La API valida identidad y permisos antes de acceder a los datos

Express en capas: routes, controllers, services y repositories. Docker Compose levanta todo.

Modelo de datos e integridad

Entidades y relaciones

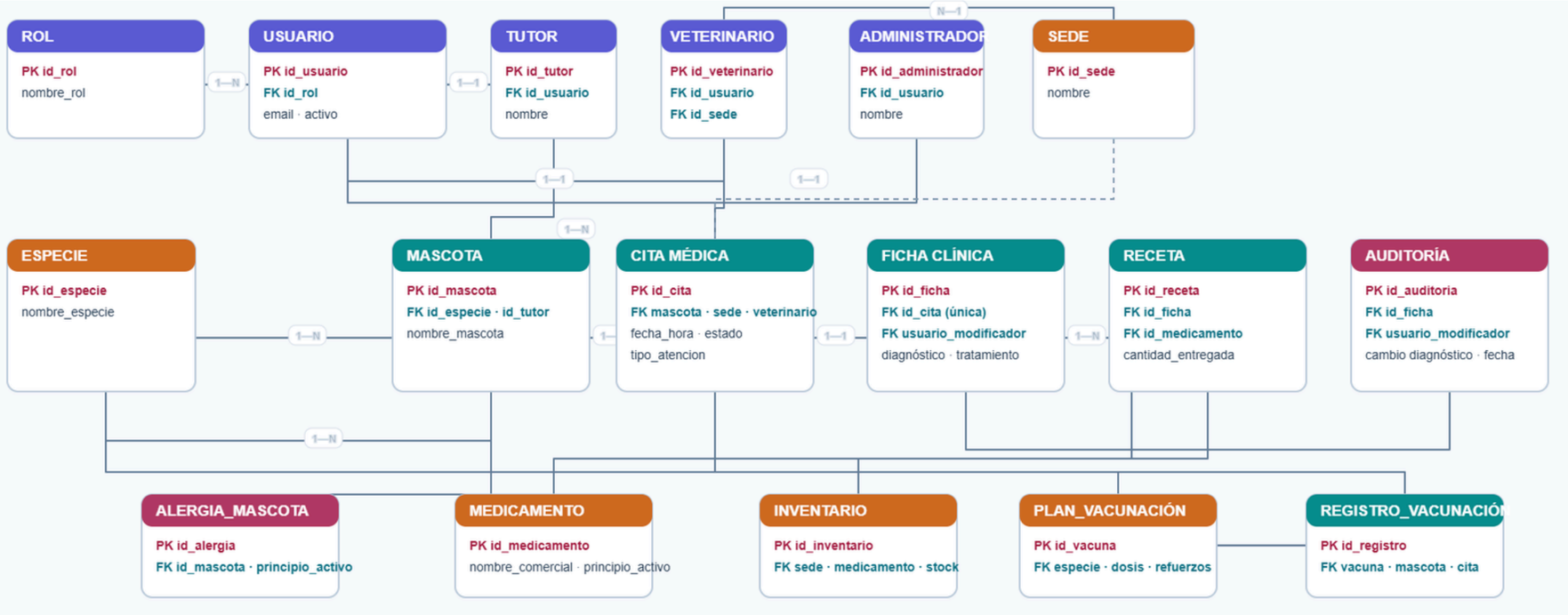
Un tutor tiene mascotas; cada cita tiene su ficha clínica y sus recetas.

Integridad del esquema

Claves foráneas, UNIQUE y CHECK: stock ⩾ 0, estados y tipos de cita válidos.

Separación de reglas

Restricciones para integridad; triggers para reglas de negocio.



Autenticación y autorización

Registro e inicio de sesión

Registro de tutores y login con JWT (vence en 8 h); contraseñas con bcrypt.

Permisos por rol

Tutor: sus mascotas. Veterinario: sus citas. Administrador: gestión y personal.

Acceso a datos propios

Un tutor que pide una ficha ajena recibe 404; un rol sin permiso, 403.



CRUD, paginación y búsquedas

Ciclo de un registro

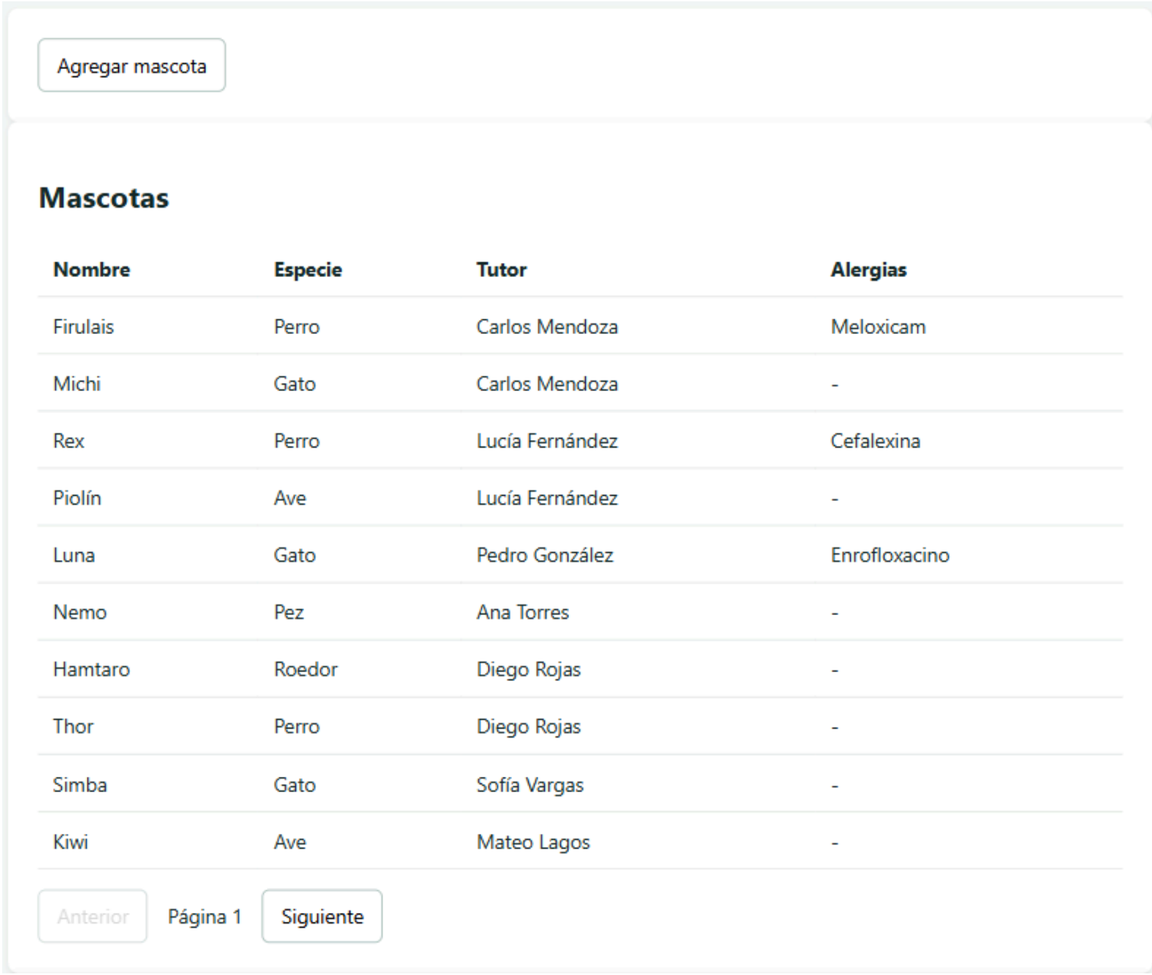
Todas las tablas se listan, consultan por id, crean, editan y borran.

Listados y filtros

Paginación con ?pagina=&limite= y filtros simples (?id_sede, ?id_mascota).

Respuesta del sistema

Los datos inválidos se rechazan con 400, 404 o 409 y un mensaje claro.



Consultas específicas del dominio

Consulta específica 1

Vacunas vencidas o por vencer en 30 días: última dosis de cada plan con refuerzos pendientes.

Consulta específica 2

Horas libres de un veterinario: bloques de 09:00 a 17:00 sin citas (generate_series).

Prueba desde la aplicación

Agendar cita usa la disponibilidad: sede → veterinario → fecha → hora libre.

```
// se buscan las vacunas vencidas o que vencen en los proximos 30 dias
export async function findVacunasPorVencer(idTutor, pagina) {
  const { rows } = await pool.query(
    `SELECT r.id_registro AS id, m.id_mascota, m.nombre_mascota AS mascota, e.nombre_especie AS especie,
            t.nombre_tutor AS tutor, u.email AS correo_tutor, p.nombre_vacuna,
            r.numero_refuerzo, p.cantidad_refuerzo, r.vencimiento_vacuna::text,
            r.vencimiento_vacuna - CURRENT_DATE AS dias_para_vencer,
            CASE WHEN r.vencimiento_vacuna < CURRENT_DATE THEN 'VENCIDA' ELSE 'POR VENCER' END AS estado
     FROM "REGISTRO_VACUNACION" r
     JOIN "MASCOTA" m ON m.id_mascota = r.id_mascota
     JOIN "ESPECIE" e ON e.id_especie = m.id_especie
     JOIN "TUTOR" t ON t.id_tutor = m.id_tutor
     JOIN "USUARIO" u ON u.id_usuario = t.id_usuario
     JOIN "PLAN_VACUNACION" p ON p.id_vacuna = r.id_vacuna
     WHERE r.numero_refuerzo < p.cantidad_refuerzo
       AND r.vencimiento_vacuna <= CURRENT_DATE + 30
       -- solo la ultima dosis de cada plan: no existe otra dosis posterior
       AND NOT EXISTS (
         SELECT 1 FROM "REGISTRO_VACUNACION" posterior
         WHERE posterior.id_mascota = r.id_mascota
           AND posterior.id_vacuna = r.id_vacuna
           AND posterior.numero_refuerzo > r.numero_refuerzo
       )
       AND ($1::int IS NULL OR m.id_tutor = $1)
     ORDER BY r.vencimiento_vacuna
     LIMIT $2 OFFSET $3`,
    [idTutor, pagina.limite, pagina.offset],
  );
  return rows;
}
```

```
// se calculan las horas libres: bloques de 09:00 a 17:00 sin citas de ese veterinario
export async function findBloquesDisponibles(idVeterinario, fecha, pagina) {
  const { rows } = await pool.query(
    `SELECT to_char(bloque, 'HH24:MI') AS hora_disponible,
            to_char(bloque, 'YYYY-MM-DD HH24:MI') AS fecha_hora_exacta
     -- generate_series crea una fila por cada hora del dia
     FROM generate_series($2::date + TIME '09:00', $2::date + TIME '17:00', INTERVAL '1 hour') AS bloque
     WHERE NOT EXISTS (
       SELECT 1 FROM "CITA_MEDICA" c
       WHERE c.id_veterinario = $1
         AND c.estado <> 'Cancelada'
         AND date_trunc('hour', c.fecha_hora) = bloque
     )
     ORDER BY bloque
     LIMIT $3 OFFSET $4`,
    [idVeterinario, fecha, pagina.limite, pagina.offset],
  );
  return rows;
}
```

Procedimientos y transacciones

Procedimiento 1

sp_registrar_atencion: cierra la ficha, receta y descuenta el stock de la sede.

Atomicidad

Un solo CALL: con alergia o sin stock no se guarda nada (probado).

```
lab1bd_veterinaria=# -- Estado ANTES: cita, ficha e inventario del medicamento 2
lab1bd_veterinaria=# SELECT c.id_cita, c.estado, f.id_ficha, f.estado AS ficha_abierta
lab1bd_veterinaria-# FROM "CITA_MEDICA" c
lab1bd_veterinaria-# JOIN "FICHA_CLINICA" f ON f.id_cita = c.id_cita
lab1bd_veterinaria-# WHERE c.id_cita = 17;
 id_cita |   estado    | id_ficha | ficha_abierta
---------+-------------+----------+---------------
      17 | Programada |       16 | t
(1 row)

lab1bd_veterinaria=#
lab1bd_veterinaria=# SELECT i.id_sede, m.nombre_comercial, i.stock_actual
lab1bd_veterinaria-# FROM "INVENTARIO" i
lab1bd_veterinaria-# JOIN "MEDICAMENTO" m ON m.id_medicamento = i.id_medicamento
lab1bd_veterinaria-# WHERE i.id_sede = 1 AND i.id_medicamento = 2;
 id_sede | nombre_comercial | stock_actual
---------+------------------+--------------
       1 | Cefapet 500mg    |           40
(1 row)
```

```
lab1bd_veterinaria=# BEGIN;
BEGIN
lab1bd_veterinaria=*#
lab1bd_veterinaria=*# CALL sp_registrar_atencion(
lab1bd_veterinaria(*#   17,
lab1bd_veterinaria(*#   'Control sano',
lab1bd_veterinaria(*#   'Vacunación y revisión general',
lab1bd_veterinaria(*#   1,
lab1bd_veterinaria(*#   ARRAY[2]::INT[],
lab1bd_veterinaria(*#   ARRAY[2]::INT[],
lab1bd_veterinaria(*#   2
lab1bd_veterinaria(*# );
```

```
lab1bd_veterinaria=*# -- Cita completada y ficha cerrada
lab1bd_veterinaria=*# SELECT c.id_cita, c.estado, f.id_ficha, f.diagnostico, f.estado AS ficha_abierta
lab1bd_veterinaria-*# FROM "CITA_MEDICA" c
lab1bd_veterinaria-*# JOIN "FICHA_CLINICA" f ON f.id_cita = c.id_cita
lab1bd_veterinaria-*# WHERE c.id_cita = 17;
 id_cita |   estado    | id_ficha | diagnostico  | ficha_abierta
---------+-------------+----------+--------------+---------------
      17 | Completada |       16 | Control sano | f
(1 row)

lab1bd_veterinaria=*#
lab1bd_veterinaria=*# -- Receta creada e inventario descontado
lab1bd_veterinaria=*# SELECT r.id_receta, r.id_ficha, m.nombre_comercial, r.cantidad_entregada,
lab1bd_veterinaria-*#        i.stock_actual AS stock_actual
lab1bd_veterinaria-*# FROM "RECETA" r
lab1bd_veterinaria-*# JOIN "MEDICAMENTO" m ON m.id_medicamento = r.id_medicamento
lab1bd_veterinaria-*# JOIN "INVENTARIO" i ON i.id_medicamento = r.id_medicamento AND i.id_sede = 1
lab1bd_veterinaria-*# WHERE r.id_ficha = (SELECT id_ficha FROM "FICHA_CLINICA" WHERE id_cita = 17);
 id_receta | id_ficha | nombre_comercial | cantidad_entregada | stock_actual
-----------+----------+------------------+--------------------+--------------
         6 |       16 | Cefapet 500mg    |                  2 |           38
(1 row)

lab1bd_veterinaria=*#
lab1bd_veterinaria=*# -- Dosis aplicada y citas de refuerzo generadas
lab1bd_veterinaria=*# SELECT id_registro, id_vacuna, id_mascota, id_cita, numero_refuerzo
lab1bd_veterinaria-*# FROM "REGISTRO_VACUNACION"
lab1bd_veterinaria-*# WHERE id_cita = 17;
 id_registro | id_vacuna | id_mascota | id_cita | numero_refuerzo
-------------+-----------+------------+---------+-----------------
           7 |         2 |          1 |      17 |               0
(1 row)

lab1bd_veterinaria=*#
lab1bd_veterinaria=*# SELECT id_cita, fecha_hora, estado
lab1bd_veterinaria-*# FROM "CITA_MEDICA"
lab1bd_veterinaria-*# WHERE id_mascota = 1
lab1bd_veterinaria-*# ORDER BY fecha_hora DESC
lab1bd_veterinaria-*# LIMIT 3;
 id_cita |     fecha_hora      |   estado
---------+---------------------+------------
      23 | 2027-01-04 10:00:00 | Programada
      22 | 2026-12-05 10:00:00 | Programada
      17 | 2026-11-05 10:00:00 | Completada
```

Triggers de validación y auditoría

Regla de negocio

trg_verificar_alergia: bloquea la receta si la mascota es alérgica.

Auditoría de diagnósticos

trg_auditoria_ficha: diagnóstico anterior, nuevo, usuario y fecha.

Prueba de ambos casos

Firulais + Meloxicam se rechaza; corregir un diagnóstico queda auditado.

```
-- se ejecuta antes de guardar cada receta
CREATE TRIGGER trg_verificar_alergia
BEFORE INSERT OR UPDATE ON "RECETA"
FOR EACH ROW
EXECUTE FUNCTION fn_validar_alergia();
```

```
CREATE OR REPLACE FUNCTION fn_validar_alergia()
RETURNS TRIGGER AS $$
DECLARE
    v_principio_activo VARCHAR;
    v_id_mascota INT;
    v_alergia_existe INT;
BEGIN
    SELECT principio_activo INTO v_principio_activo
    FROM "MEDICAMENTO" WHERE id_medicamento = NEW.id_medicamento;

    SELECT c.id_mascota INTO v_id_mascota
    FROM "FICHA_CLINICA" f
    JOIN "CITA_MEDICA" c ON f.id_cita = c.id_cita
    WHERE f.id_ficha = NEW.id_ficha;

    SELECT COUNT(*) INTO v_alergia_existe
    FROM "ALERGIA_MASCOTA"
    WHERE id_mascota = v_id_mascota AND principio_activo = v_principio_activo;

    IF v_alergia_existe > 0 THEN
        RAISE EXCEPTION 'Operación denegada: La mascota es alérgica al principio activo %', v_principio_activo;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Índices y vista materializada

Tres columnas críticas

CITA_MEDICA: id_mascota, fecha_hora e (id_veterinario, fecha_hora).

Vista materializada

Atenciones y diagnósticos frecuentes por especie y sede (12 meses).

Refresco de resultados

node-cron la refresca a las 03:00, o a mano; el índice único permite CONCURRENTLY.

```
 nombre_sede | especie | cantidad_atenciones | diagnosticos_mas_frecuentes
-------------+---------+---------------------+-----------------------------
 Sede Centro | Gato    |                   2 | Gastroenteritis aguda
 Sede Norte  | Perro   |                   5 | Control anual preventivo
 Sede Sur    | Ave     |                   2 | Chequeo preventivo aviar
 Sede Sur    | Roedor  |                   1 | Sobrecrecimiento dental
(4 rows)
```

Fragmentos de código:

```sql
-- procedimiento 2: se aplica una dosis de un plan y, si es la primera, se agendan los refuerzos
CREATE OR REPLACE PROCEDURE sp_aplicar_vacuna(
    p_id_cita INT,
    p_id_vacuna INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    -- %ROWTYPE: variable con la forma de una fila completa de la tabla
    v_cita "CITA_MEDICA"%ROWTYPE;
    v_plan "PLAN_VACUNACION"%ROWTYPE;
    v_dosis_previas INT;
BEGIN
    -- se busca la cita
    SELECT * INTO v_cita FROM "CITA_MEDICA" WHERE id_cita = p_id_cita;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'La cita % no existe.', p_id_cita;
    END IF;

    -- se revisa que la cita no este cancelada ni tenga ya esta vacuna
    IF v_cita.estado = 'Cancelada' THEN
        RAISE EXCEPTION 'La cita % está cancelada.', p_id_cita;
    END IF;

    IF EXISTS (SELECT 1 FROM "REGISTRO_VACUNACION" WHERE id_cita = p_id_cita AND id_vacuna = p_id_vacuna) THEN
        RAISE EXCEPTION 'Esta vacuna ya se registró en la cita %.', p_id_cita;
    END IF;

    -- se busca el plan y se revisa que sea de la especie de la mascota
    SELECT p.* INTO v_plan
    FROM "PLAN_VACUNACION" p
    JOIN "MASCOTA" m ON m.id_especie = p.id_especie
    WHERE p.id_vacuna = p_id_vacuna AND m.id_mascota = v_cita.id_mascota;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El plan de vacunación no existe o no es para la especie de la mascota.';
    END IF;

    -- se cuentan las dosis previas para saber cual toca
    SELECT COUNT(*) INTO v_dosis_previas
    FROM "REGISTRO_VACUNACION"
    WHERE id_mascota = v_cita.id_mascota AND id_vacuna = p_id_vacuna;

    IF v_dosis_previas > v_plan.cantidad_refuerzo THEN
        RAISE EXCEPTION 'La mascota ya completó el plan %.', v_plan.nombre_vacuna;
    END IF;

    -- se registra la dosis en la fecha de la cita
    INSERT INTO "REGISTRO_VACUNACION" (id_vacuna, id_mascota, id_cita, fecha_aplicacion, vencimiento_vacuna, numero_refuerzo)
    VALUES (p_id_vacuna, v_cita.id_mascota, p_id_cita,
            v_cita.fecha_hora::date, v_cita.fecha_hora::date + v_plan.dias_refuerzo, v_dosis_previas);

    -- si es la primera dosis, se agenda una cita por refuerzo (generate_series da n = 1, 2...) y a cada una se le crea su ficha
    IF v_dosis_previas = 0 THEN
        WITH nuevas AS (
            INSERT INTO "CITA_MEDICA" (id_mascota, id_sede, id_veterinario, tipo_atencion, fecha_hora, estado)
            SELECT v_cita.id_mascota, v_cita.id_sede, v_cita.id_veterinario, v_cita.tipo_atencion,
                   v_cita.fecha_hora + n * v_plan.dias_refuerzo * INTERVAL '1 day', 'Programada'
            FROM generate_series(1, v_plan.cantidad_refuerzo) AS n
            RETURNING id_cita
        )
        INSERT INTO "FICHA_CLINICA" (id_cita)
        SELECT id_cita FROM nuevas;
    END IF;
END;
```

```sql
-- se ejecuta antes de guardar cada receta
CREATE TRIGGER trg_verificar_alergia
BEFORE INSERT OR UPDATE ON "RECETA"
FOR EACH ROW
EXECUTE FUNCTION fn_validar_alergia();
```

```sql
CREATE OR REPLACE FUNCTION fn_validar_alergia()
RETURNS TRIGGER AS $$
DECLARE
    v_principio_activo VARCHAR;
    v_id_mascota INT;
    v_alergia_existe INT;
BEGIN
    SELECT principio_activo INTO v_principio_activo
    FROM "MEDICAMENTO" WHERE id_medicamento = NEW.id_medicamento;

    SELECT c.id_mascota INTO v_id_mascota
    FROM "FICHA_CLINICA" f
    JOIN "CITA_MEDICA" c ON f.id_cita = c.id_cita
    WHERE f.id_ficha = NEW.id_ficha;

    SELECT COUNT(*) INTO v_alergia_existe
    FROM "ALERGIA_MASCOTA"
    WHERE id_mascota = v_id_mascota AND principio_activo = v_principio_activo;

    IF v_alergia_existe > 0 THEN
        RAISE EXCEPTION 'Operación denegada: La mascota es alérgica al principio activo %', v_principio_activo;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Instalación y entrega reproducible

Repositorio organizado

backend/, frontend/, database/ (01 a 06) y docs/ (dump y pruebas).

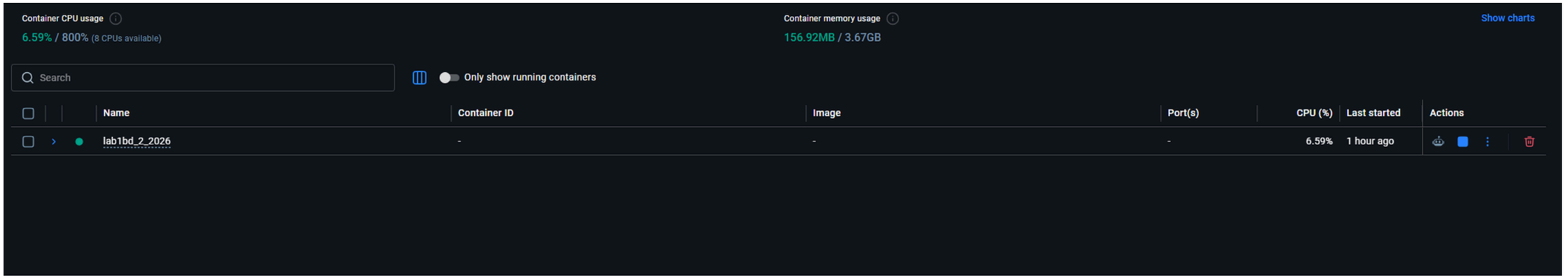
Base de datos reproducible

Docker crea el esquema y carga los datos al primer arranque; además, un dump.

README ejecutable

Copiar el .env, docker compose up --build -d y abrir localhost:5173.

```
jjmca91000@DESKTOP-J3QQU1G:~/universidad/proyectos/GitHub/LAB1BD_2_2026$ cp backend/.env.example backend/.env
jjmca91000@DESKTOP-J3QQU1G:~/universidad/proyectos/GitHub/LAB1BD_2_2026$ docker compose up --build -d
```



Contribuciones del equipo

Integrante	Aporte concreto
Antony Huahuisa	Dio su github el dia antes
Nicolas Orellana	Presentacion, organizacion de la informacion
Francisco Briones	Backend, querys, etc
Melissa Shimada	Tester principal, verifico el funcionamiento, corrigió errores.
Juan Campos	Frontend, forma de mostrar los datos, vistas, etc.
Ignacio Caro	Base de datos, esquemas, sql, triggers, procedimientos.

Conclusiones del laboratorio

Resultado demostrado

Una atención (receta, stock y vacuna con refuerzos) se guarda entera en una transacción, o nada.

Decisión técnica y evidencia

La lógica crítica vive en PostgreSQL; 139 pruebas de la API

Aprendizaje del equipo

Validar en cada capa (base, API e interfaz) evita errores que una sola capa no detecta.